

HW4: Model Validation Report

建议 1: 数据泄漏问题

该组代码可能混淆了训练集、验证集与测试集之间的关系，没有将训练集和验证集先分开，导致了数据泄漏问题。

1.1 填充缺失值过程中产生了数据泄漏

UnifiedDataProcessor 类的 fill_missing_values 方法中，代码直接用当前数据集（含验证集）的中位数填充数值列、众数填充分类列，这使得验证集的分布信息提前被模型接触，造成较为严重的数据泄露问题，可能导致 in-sample MAE 偏低。应该先基于训练集计算中位数、众数等统计量，再将其应用于验证集的缺失值填充，避免验证集数据参与统计量计算。

```
def fill_missing_values(self):
    """填充缺失值"""
    for df in [self.train_data, self.test_data]:
        numeric_cols = df.select_dtypes(include=[np.number]).columns
        categorical_cols = df.select_dtypes(exclude=[np.number]).columns

        # 数值列用中位数填充
        df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())

        # 分类列用众数填充
        for col in categorical_cols:
            if not df[col].mode().empty:
                df[col] = df[col].fillna(df[col].mode()[0])
            else:
                df[col] = df[col].fillna("未知")
```

1.2 KMeans 方法中存在数据泄漏

首先，在 train_kmeans_model 函数中，KMeans 是在原始数据中进行训练的，没有区分训练集和验证集，导致数据泄漏问题；其次，在 add_advanced_features 方法中，租金数据直接复用了房价训练出的 KMeans 模型。房价和租金的地理分布可能存在差异，聚类标签不具备迁移性，所以应该分开训练并运用。

```
def train_kmeans_model(data_path, n_clusters=20):
    """训练KMeans聚类模型"""
    data = pd.read_csv(data_path, low_memory=False)
    X = data[['lon', 'lat']].dropna()

    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    kmeans.fit(X)

    # 保存模型
    os.makedirs('./src/model', exist_ok=True)
    joblib.dump(kmeans, './src/model/kmeans_model.pkl')

    print(f"KMeans模型训练完成，聚类数: {n_clusters}")
    return kmeans
```

```
def add_advanced_features(self, df):
    """添加高级特征"""
    # 朝向得分
    if '房屋朝向' in df.columns:
        df['朝向得分'] = df['房屋朝向'].apply(self.calculate_orientation_score)

    # 聚类特征
    if all(col in df.columns for col in ['lon', 'lat']):
        try:
            df['cluster_label'] = self.kmeans.predict(df[['lon', 'lat']])
        except:
            df['cluster_label'] = 0
```

1.3 标准化过程中产生了数据泄漏

首先，代码中进行了多次重复的标准化，部分标准化没有区分类别变量和数值变量。其次，UnifiedModel 的 cross_validate 方法在全数据集上拟合 StandardScaler，导致验证集信息混入训练过程，造成了数据泄漏。

```
def cross_validate(self, X, y, cv=6):
    """交叉验证"""
    from sklearn.model_selection import cross_val_score

    if self.use_log:
        y_cv = np.log1p(y)
    else:
        y_cv = y

    if self.model_type != 'log':
        X_cv = self.scaler.fit_transform(X)
    else:
        X_cv = X
```

建议 2：数据处理问题

2.1 填充缺失值数据泄露

UnifiedDataProcessor 类的 fill_missing_values 方法中，代码通过循环同时处理训练集和测试集，直接用当前数据集（含测试集）的中位数填充数值列、众数填充分类列，这使得测试集的分布信息提前被模型接触，造成较为严重的数据泄露问题。应该先基于训练集计算中位数、众数等统计量，再将其应用于测试集的缺失值填充，避免测试集数据参与统计量计算。

```
def fill_missing_values(self):
    """填充缺失值"""
    for df in [self.train_data, self.test_data]:
        numeric_cols = df.select_dtypes(include=[np.number]).columns
        categorical_cols = df.select_dtypes(exclude=[np.number]).columns

        # 数值列用中位数填充
        df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())

        # 分类列用众数填充
        for col in categorical_cols:
            if not df[col].mode().empty:
                df[col] = df[col].fillna(df[col].mode()[0])
            else:
                df[col] = df[col].fillna("未知")
```

2.2 梯户比例拆分过程存在缺陷

split_ladder_household 方法中，中文数字映射仅支持“零”到“十”，超过“十”的数字（如“十一”“十二”“二十”）会被识别为 0，导致特征提取错误。

```
def split_ladder_household(self, df):
    """拆分梯户比例"""
    if '梯户比例' in df.columns:
        def split_ratio(ratio_str):
            if pd.isna(ratio_str):
                return pd.Series([0, 0])
            ratio_str = str(ratio_str).strip()
            chinese_map = {'零':0, '一':1, '二':2, '三':3, '四':4, '五':5, '六':6, '七':7, '八':8, '九':9, '十':10}

            ladder_part = re.split(r'梯', ratio_str)[0] if '梯' in ratio_str else ''
            ladder = chinese_map.get(ladder_part, 0)
            household = 0

            if '梯' in ratio_str:
                household_part = re.split(r'梯', ratio_str)[1]
                household_part = re.split(r'户', household_part)[0] if '户' in household_part else household_part
                household = chinese_map.get(household_part, 0)
            return pd.Series([ladder, household])

        ladder_household = df['梯户比例'].apply(split_ratio)
        ladder_household.columns = ['梯num', '户num']
        df = pd.concat([df, ladder_household], axis=1)
        df = df.drop(columns=['梯户比例'])
    return df
```

2.3 停车费用数据处理有误

停车费用中存在多个数字，也存在不同的单位（月租时租），此处是直接提取正则中的第一个数据（re.findall()[0]），可能忽略范围或者导致单位不一。

```
def extract_numeric_from_unit(self, df):
    """从带单位文本中提取数值"""
    unit_cols = ['燃气费', '供热费', '停车费用', '物业费', '房屋总数', '楼栋总数', '绿化率', '建筑面积', '面积']

    for col in unit_cols:
        if col in df.columns:
            df[col] = df[col].apply(
                lambda x: float(re.findall(r'[+]?\\d+\\.?\\d*', str(x))[0])
                if re.findall(r'[+]?\\d+\\.?\\d*', str(x)) else np.nan
            )
    return df
```

建议 3：特征工程问题

3.1 建筑面积处理时出现多重共线性（原始特征与 log 变换特征共存）

建筑面积与对数变换后的 log_建筑面积同时保留在数据集中，线性模型对多重共线性敏感，会导致系数估计不稳定、模型解释性变差。

```
# 对数变换
area_col = '建筑面积' if '建筑面积' in df.columns else '面积'
if area_col in df.columns:
    df['log_' + area_col] = np.log(df[area_col].replace(0, 0.1))

if self.target in df.columns:
    df['log_' + self.target] = np.log(df[self.target])

return df
```

3.2 特征值筛选有隐患

此处是根据训练集的缺失比例删除特征，并且保留关键列，然后根据训练集中筛出的列，在测试集中完全对齐训练集的列结构。这可能导致测试集中原本信息充足的变量因为在训练集中缺失率略高而被删除，丢失有用信息。同时，将删除的阈值设置为 0.2 也过于严格，可能丢失大量有用的列和信息。建议放宽阈值，多进行缺失值的填补而非直接删除。

```
# 筛选特征（基于缺失率）
if df_name == 'train_data':
    missing_ratio = df.isnull().mean()
    self.keep_cols = missing_ratio[missing_ratio <= self.MISSING_THRESHOLD].index.tolist()
    essential_cols = [self.target, 'ID', 'text_combined_raw', '室num', '厅num', '厨num', '卫num']
    self.keep_cols = list(set(self.keep_cols + [c for c in essential_cols if c in df.columns]))
    df = df[self.keep_cols]
else:
    # 测试集对齐训练集特征
    for col in self.keep_cols:
        if col not in df.columns:
            df[col] = np.nan
    df = df[self.keep_cols]
```

3.3 Bert 函数调用仍可继续优化

代码中加载了 BERT 模型，但在文本特征处理环节调用不充分，可能会造成启动时间长的的问题。同时，原代码中使用了函数 AutoModelForSequenceClassification，可以尝试使用特征提取模型(BertModel)。

```
# 加载模型
try:
    self.kmeans = joblib.load('./src/model/kmeans_model.pkl')
    self.model = AutoModelForSequenceClassification.from_pretrained("jackietung/bert-base-chinese-sentiment-finetuned")
    self.tokenizer = AutoTokenizer.from_pretrained("jackietung/bert-base-chinese-sentiment-finetuned")
except:
    print("部分模型加载失败, 将继续使用基础功能")
```

同时，小组代码仅对于“房屋优势”等文本特征，仅使用简单的、已确定的关键词匹配进行二值化，未能有效利用加载的 NLP 模型的语义理解能力，无法挖掘新的语义特征。

```
def process_text_features(self, df):
    """处理文本特征"""
    text_cols = ['客户反馈', '房屋优势', '核心卖点', '周边配套', '交通出行']
    text_cols = [c for c in text_cols if c in df.columns]

    if text_cols:
        df['text_combined_raw'] = df[text_cols].apply(lambda x: " ".join(x.astype(str)), axis=1)

        for col in text_cols:
            for kw in ['地铁', '学区', '公园', '医院', '满五年', '精装', '南北通透']:
                df[f'{col}_has_{kw}'] = df[col].apply(lambda x: 1 if kw in str(x) else 0)

        df = df.drop(columns=text_cols, errors='ignore')

    return df
```

建议 4：交叉验证计算问题

4.1 交叉验证 mae 没有准确还原

小组代码此处处¹在 $\log(y+1)$ 基础上计算 cv_mae ，但是在原始尺度上计算的 cv_mae 和在 $\log(y+1)$ 上计算的 cv_mae 之间并没有近似的可逆关系，导致交叉验证 mae 报告存在偏误。

```
def cross_validate(self, X, y, cv=6):
    """交叉验证"""
    from sklearn.model_selection import cross_val_score

    if self.use_log:
        y_cv = np.log1p(y)
    else:
        y_cv = y

    if self.model_type != 'lgb':
        X_cv = self.scaler.fit_transform(X)
    else:
        X_cv = X

    scores = cross_val_score(self.model, X_cv, y_cv, cv=cv, scoring='neg_mean_absolute_error')
    cv_mae = -scores.mean()

    if self.use_log:
        # 近似转换回原始尺度
        cv_mae = np.exp1(cv_mae)
```